

# 大数据计算及应用

---

## 推荐系统1

从<http://www.mmds.org>改编的幻灯片

# 议程

## 高维数据

位置敏感散列

聚类

维度减少

## 图形数据

PageRank, SimRank

社区检测

垃圾邮件检测

## 无限数据

过滤数据流

网络广告

对流的查询

## 机器学习

SVM

决策树

感知器、kNN

## 附录

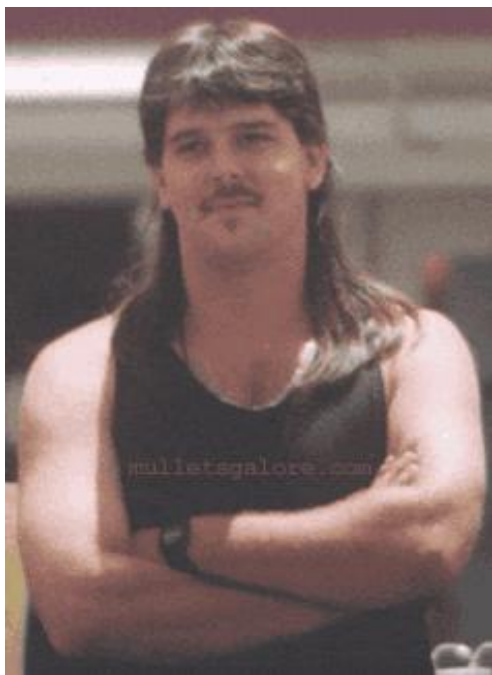
推荐系统

关联规则

重复文档检测

# 示例：推荐系统

---



## □ 客户X

- 购买Metallica CD
- 购买Megadeth CD

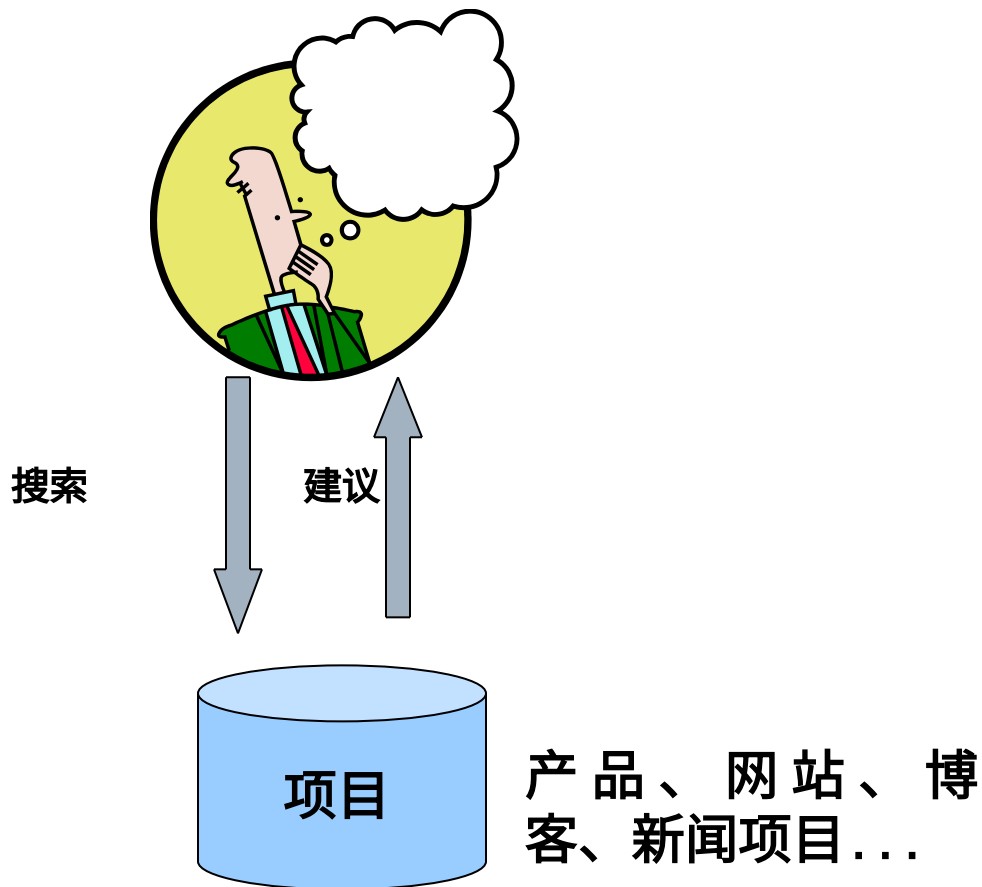


## □ 客户Y

- 是否搜索Metallica
- 根据收集到的数据，系统推荐Megadeth

客户X

# 建议



示例：

amazon.com



movielens  
helping you find the *right* movies



# 从稀缺到丰富

---

- 货架空间对传统零售商来说是一种稀缺商品
  - 还有：电视网络、电影院、...
- 网络使产品信息的传播成本接近于零
  - 从稀缺到丰富
- 更多选择需要更好的过滤器
  - 推荐引擎
  - 《How Into Thin Air》是如何让《Touching the Void》成为畅销书的：<http://www.wired.com/wired/archive/12.10/tail.html>

# 旁注：长尾



Sources: Erik Brynjolfsson and Jeffrey Hu, MIT, and Michael Smith, Carnegie Mellon; Barnes & Noble; Netflix; RealNetworks

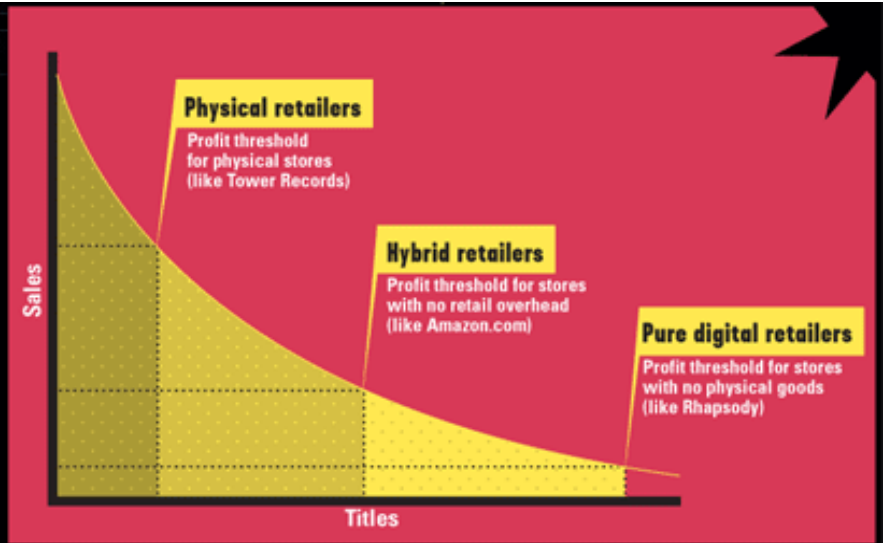
资料来源：Chris Anderson (2004年)

# 实体与在线

## THE BIT PLAYER ADVANTAGE

Beyond bricks and mortar there are two main retail models – one that gets halfway down the Long Tail and another that goes all the way. The first is the familiar hybrid model of Amazon and Netflix, companies that sell physical goods online. Digital catalogs allow them to offer unlimited selection along with search, reviews, and recommendations, while the cost savings of massive warehouses and no walk-in customers greatly expands the number of products they can sell profitably.

Pushing this even further are pure digital services, such as iTunes, which offer the additional savings of delivering their digital goods online at virtually no marginal cost. Since an extra database entry and a few megabytes of storage on a server cost effectively nothing, these retailers have no economic reason not to carry *everything* available.



## “IF YOU LIKE BRITNEY, YOU’LL LOVE ...”

Just as lower prices can entice consumers down the Long Tail, recommendation engines drive them to obscure content they might not find otherwise.



Amazon sales rank

Source: Amazon.com

阅读<http://www.wired.com/wired/archive/12.10/tail.html>了解更多！



# 建议类型

---

## □ 编辑和手工策划

- 收藏列表
- “必需”物品清单

## □ 简单聚合

- 排行榜前十，最受欢迎，最近上传

## □ 针对个人用户定制

- 亚马逊，Netflix，...

# 正式模型

---

□  $X$ =客户群

□  $S$ =项目集

□ 效用函数  $u : X \times S \rightarrow R$

- $R$ =评分集

- $R$ 是一个全序集

- 例如，0-5星， $[0,1]$ 中的实数

# 公用事业矩阵

---

《阿凡达LOTR矩阵海盗》Matrix Pirates

爱丽丝1

0.2

Bob  
鲍勃

0.5

0.3

Carol  
卡罗尔0.2

1

David  
大卫

0.4

# 关键问题

---

## □ (1)收集矩阵的“已知”评分

- 如何在效用矩阵中收集数据

## □ (2)根据已知的评分推断未知的评分

- 主要对未知评分较高的作品感兴趣
  - 我们不感兴趣的是知道你不喜欢什么，而是你喜欢什么

## □ (3)外推方法评价

- 如何衡量推荐方法的成功/性能

# (1) 收集评分

---

## □ 显式

- 请用户对商品进行评分
- 在实践中效果不好，人们不会被打扰

## □ 隐含

- 根据用户操作学习评分
  - 例如，购买意味着高评级
- 低评级怎么办？

## (2) 推算公用事业

---

- 关键问题：效用矩阵 $U$ 是稀疏的
  - 大多数人尚未对大多数商品进行评分
  - 冷启动：
    - 新商品暂无评分
    - 新用户没有历史记录
  
- 推荐系统有三种方法：
  - 1) 基于内容
  - 2) 协作 } 今天
  - 3) 基于潜在因素

# 基于内容 推荐系统

---

# 基于内容的推荐

---

- 主要想法：向客户 $x$ 推荐与之前被 $x$ 高度评价的项目相似的项目

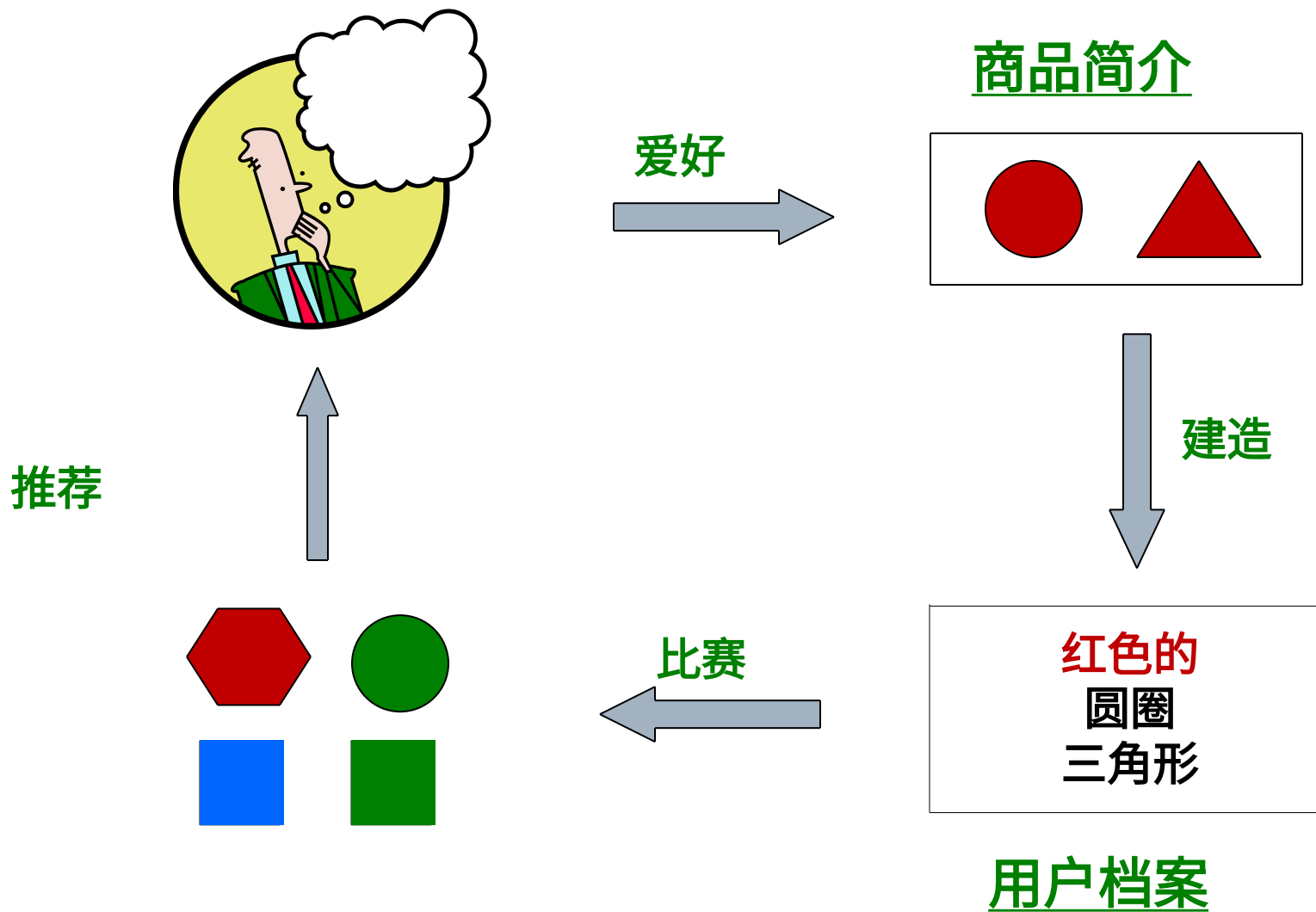
## 样例

- 电影推荐
  - 推荐演员（或导演）、类型相同电影
- 网站、博客、新闻
  - 推荐其他具有“相似”内容的网站



# 行动计划

---



# 项目简介

---

□ 为每个项目创建一个项目简介

□ 特征集（向量）

- 电影：作者、片名、演员、导演、...
- 文本：文档中“重要”词汇的集合

□ 如何选择重要的特征？

- 文本挖掘中常用的启发式方法是TF-IDF (Term frequency \* Inverse Doc Frequency)
  - 术语...特性
  - 文档...项目

## 附注：TF-IDF

---

$f_{ij}$  = 文档（项目） $j$  中术语（特征） $i$  的频率

$$TF_{ij} = \frac{f_{ij}}{\max_k f_{kj}}$$

注：我们对TF进行标准化

对“更长”产品给予折扣

提及术语 $i$ 的文档数为 $n_i$ ，文档总数为 $N$

$$IDF_i = \log \frac{N}{n_i}$$

TF-IDF得分： $w_{ij} = TF_{ij} \times IDF_i$

文档配置文件=具有最高TF-IDF分数的单词集合，以及它们的分数

# 用户档案和预测

---

## □ 用户配置文件的可能性：

- 评级项目概况的加权平均值
- 变化：与项目平均评级的差异重量
- ...

## □ 预测启发式：

- 给定用户配置文件x和项目配置文件i，进行估计

$$u(x, i) = \cos(x, i) = \frac{x \cdot i}{\|x\| \cdot \|i\|}$$

# 优点：基于内容的方法

---

## □ +：无需其他用户数据

- 无冷启动或稀疏性问题

## □ +：能够向有独特品味的用户推荐

## □ +：能够推荐新、不受欢迎的商品

- 没有一级评定者问题

## □ +：能够提供解释

- 可以通过列出导致推荐项目的特征内容来解释推荐项目

# 缺点：基于内容的方法

---

## □ - ：找到合适的特征是困难的

- 例如，图像、电影、音乐

## □ - ：新用户建议

- 如何建立用户档案？

## □ -：Overspecialization

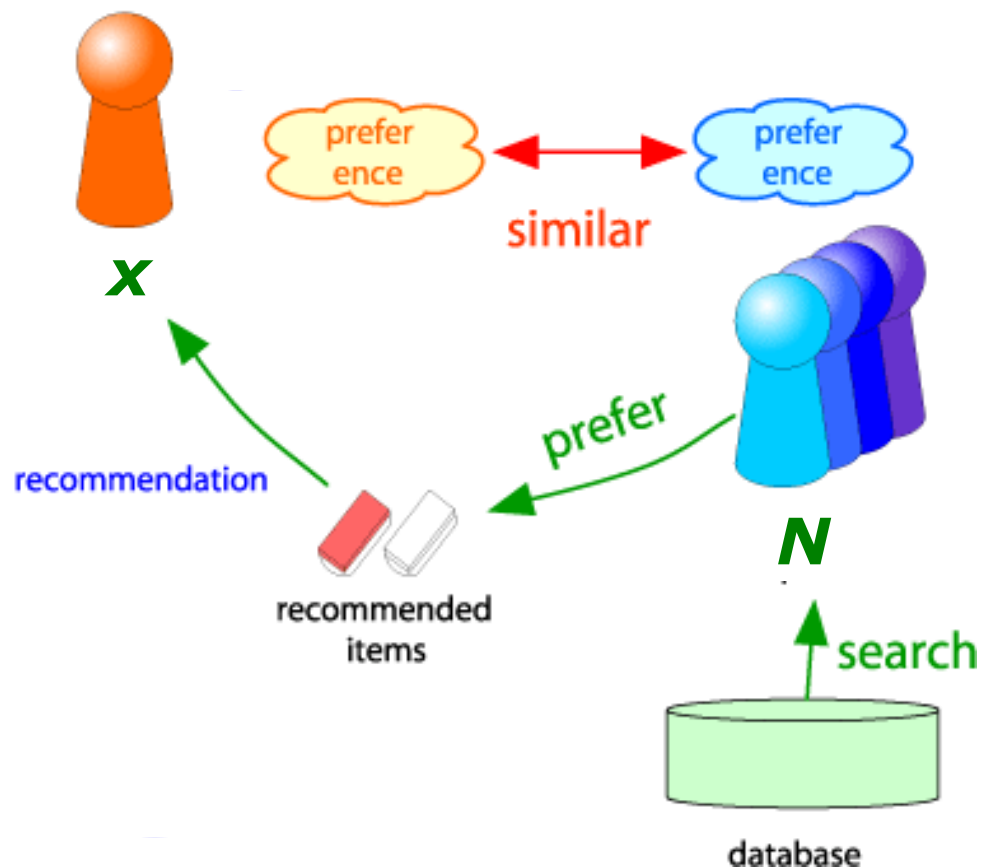
- 不推荐用户内容档案以外的项目
- 用户可能有多个兴趣
- 无法利用其他用户的质量判断

# 协同过滤

利用其他用户的质量判断

# 协同过滤

- 考虑用户 $x$
- 查找其他用户的集合 $N$ ，这些用户的评分与 $x$ 的评分“相似”
- 根据 $N$ 中的用户评分估计 $x$ 的评分





# 查找“相似”用户

□ LJeat rxccar bdestheim lvectorarity mofeausersurex

的评分  $r_x = [*, \_, \_, *, ***]$   
 $r_y = [*, \_, **, **, \_]$

■ 问题：忽略评分值

## □ 余弦相似度量

■  $\text{sim}(\mathbf{x}, \mathbf{y}) = \cos(\mathbf{r}_x, \mathbf{r}_y) = \frac{\mathbf{r}_x \cdot \mathbf{r}_y}{\|\mathbf{r}_x\| \cdot \|\mathbf{r}_y\|}$

■ 问题：将缺失的评分视为“负面”

## □ 皮尔逊相关系数

■ 用户x和y均评价过的  $S_{xy}$  = 项目

$r_x, r_y$  作为  
集合： $r_x = \{1, 4, 5\}$

$r_y = \{1, 3, 4\}$

$r_x, r_y$  作为点：  
 $r_x = \{1, 0, 0, 1, 3\}$   $r_y = \{1, 0, 2, 2, 0\}$

$r_x, r_y \dots \text{avg.}$

x、y的等级

$$\mathit{sim}(x, y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)(r_{ys} - \bar{r}_y)}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \bar{r}_x)^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \bar{r}_y)^2}}$$


---

# 相似度量

|   | HP1 | HP2 | HP3 | TW | SW1 | SW2 | SW3 |
|---|-----|-----|-----|----|-----|-----|-----|
| A | 4   |     |     | 5  | 1   |     |     |
| B | 5   | 5   | 4   |    |     |     |     |
| C |     |     |     | 2  | 4   | 5   |     |
| D |     | 3   |     |    |     |     | 3   |

□ 直观上，我们希望： $\text{sim}(A, B) > \text{sim}(A, C)$

□ Jaccard相似性： $1/5 < 2/4$

□ 余弦相似度： $0.386 > 0.322$

■ 将缺失的评级视为“负面”

■ 解决方案：减去（行）平均值

|   | HP1 | HP2 | HP3  | TW   | SW1  | SW2 | SW3 |
|---|-----|-----|------|------|------|-----|-----|
| A | 2/3 |     |      | 5/3  | -7/3 |     |     |
| B | 1/3 | 1/3 | -2/3 |      |      |     |     |
| C |     |     |      | -5/3 | 1/3  | 4/3 |     |
| D |     | 0   |      |      |      |     | 0   |

模拟A、B与A、C：

$0.092 > -0.559$

当数据以0为中心时，相似的余弦值是相关性

# 评级预测

---

## 从相似度度量到推荐：

- 设 $r_x$ 为用户 $x$ 的评分向量
- 设 $N$ 为与 $x$ 最相似的 $k$ 个用户集合，这些用户都对项目 $i$ 进行了评分
- 用户 $x$ 的项目 $i$ 的预测值：
  - $r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$
  - $r_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} s_{xy}}$  Shorthand:  
 $s_{xy} = \text{sim}(x, y)$
  - 其他选项？
- 还有很多其他可能的技巧...

# 项目-项目协同过滤

---

□ 到目前为止：用户-用户协同过滤

□ 另一种视图：项目-项目

- 对于项目i，查找其他类似项目
- 基于类似项目的评级估算项目i的评级
- 可以使用与用户-用户模型相同的相似度度和预测函数

$$r_{xi} = \frac{\sum_{j \in N(i; x)} s_{ij} \cdot r_{xj}}{\sum_{j \in N(i; x)} s_{ij}}$$

$s_{ij}$  ... 项目i和j的相似性

$r_{xj}$  ... 用户u对项目j的评分

$N(i; x)$  ... 由与i相似的用户评分的项目

# 项目-项目CF ( $|N|=2$ )

用户

电影

|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|----|----|----|
| 1 | 1 |   | 3 |   |   | 5 |   |   | 5 |    | 4  |    |
| 2 |   |   | 5 | 4 |   |   | 4 |   |   | 2  | 1  | 3  |
| 3 | 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3  | 5  |    |
| 4 |   | 2 | 4 |   | 5 |   |   | 4 |   |    | 2  |    |
| 5 |   |   | 4 | 3 | 4 | 2 |   |   |   |    | 2  | 5  |
| 6 | 1 |   | 3 |   | 3 |   |   | 2 |   |    | 4  |    |



-未知评分



-评级 1至5之间

# 项目-项目CF ( $|N|=2$ )

用户

电影

|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|----|----|----|
| 1 | 1 |   | 3 |   | ? | 5 |   |   | 5 |    | 4  |    |
| 2 |   |   | 5 | 4 |   |   | 4 |   |   | 2  | 1  | 3  |
| 3 | 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3  | 5  |    |
| 4 |   | 2 | 4 |   | 5 |   |   | 4 |   |    | 2  |    |
| 5 |   |   | 4 | 3 | 4 | 2 |   |   |   |    | 2  | 5  |
| 6 | 1 |   | 3 |   | 3 |   |   | 2 |   |    | 4  |    |



-用户5对电影1的估计评分

# 项目-项目CF ( $|N|=2$ )

用户

电影

|          | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|----------|---|---|---|---|---|---|---|---|---|----|----|----|
| 1        | 1 |   | 3 |   | ? | 5 |   |   | 5 |    | 4  |    |
| 2        |   |   | 5 | 4 |   |   | 4 |   |   | 2  | 1  | 3  |
| <u>3</u> | 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3  | 5  |    |
| 4        |   | 2 | 4 |   | 5 |   |   | 4 |   |    | 2  |    |
| 5        |   |   | 4 | 3 | 4 | 2 |   |   |   |    | 2  | 5  |
| <u>6</u> | 1 |   | 3 |   | 3 |   |   | 2 |   |    | 4  |    |

$\text{sim}(1,m)$

1.00

-0.18

0.41

-0.10

-0.31

0.59

邻居选择：

这里我们使用皮尔逊相关作为相似性：

识别与相似的电影

1) 从每部电影 $i$ 中减去平均评分 $m_i$

$$m_1 = (1+3+5+5+4)/5 = 3.6$$



电影1，由用户5评分

第1行： $[-2.6, 0, -0.6, 0, 0, 1.4, 0, 0, 1.4, 0, 0.4, 0]$

2) 计算各行之间的余弦相似度

---

# 项目-项目CF ( $|N|=2$ )

用户

电影

|          | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|----------|---|---|---|---|---|---|---|---|---|----|----|----|
| 1        | 1 |   | 3 |   | ? | 5 |   |   | 5 |    | 4  |    |
| 2        |   |   | 5 | 4 |   |   | 4 |   |   | 2  | 1  | 3  |
| <u>3</u> | 2 | 4 |   | 1 | 2 |   | 3 |   | 4 | 3  | 5  |    |
| 4        |   | 2 | 4 |   | 5 |   |   | 4 |   |    | 2  |    |
| 5        |   |   | 4 | 3 | 4 | 2 |   |   |   |    | 2  | 5  |
| <u>6</u> | 1 |   | 3 |   | 3 |   |   | 2 |   |    | 4  |    |

$\text{sim}(1,m)$

1.00

-0.18

0.41

-0.10

-0.31

0.59

计算相似度权重：

$$s_{1,3}=0.41, s_{1,6}=0.59$$

# 项目-项目CF ( $|N|=2$ )

用户

电影

|          | 1 | 2 | 3 | 4 | 5   | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|----------|---|---|---|---|-----|---|---|---|---|----|----|----|
| 1        | 1 |   | 3 |   | 2.6 | 5 |   |   | 5 |    | 4  |    |
| 2        |   |   | 5 | 4 |     |   | 4 |   |   | 2  | 1  | 3  |
| <u>3</u> | 2 | 4 |   | 1 | 2   |   | 3 |   | 4 | 3  | 5  |    |
| 4        |   | 2 | 4 |   | 5   |   |   | 4 |   |    | 2  |    |
| 5        |   |   | 4 | 3 | 4   | 2 |   |   |   |    | 2  | 5  |
| <u>6</u> | 1 |   | 3 |   | 3   |   |   | 2 |   |    | 4  |    |

通过加权平均值进行预测：

$$r_{1.5} = (0.41 * 2 + 0.59 * 3) / (0.41 + 0.59) = 2.6$$

$$r_{ix} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{jx}}{\sum s_{ij}}$$

# CF：通用做法

□ 定义项目 i 和 j 的相似度  $s_{ij}$  选择

□ k 个最近邻  $N(i; x)$

■ 由 x 评分的与 i 最相似的商品

□

将估计评级  $r_{xi}$  作为加权平均值：

$$r_{xi} = b_{xi} + \frac{\sum_{j \in N(i; x)} s_{ij} \cdot (r_{xj} - b_{xj})}{\sum_{j \in N(i; x)} s_{ij}}$$

$r_{xi}$  的基准估计值

$$b_{xi} = \mu + b_x + b_i$$

$\mu$  = 总体平均电影评分

$b_x$  = 用户 x 的评分偏差 = (用户 x 的平均评分) -  $\mu$

i 部影片的  $b_i$  = 评级偏差

之前

$$r_{xi} = \frac{\sum_{j \in N(i; x)} s_{ij} r_{xj}}{\sum_{j \in N(i; x)} s_{ij}}$$

# 项目-项目与用户-用户

---

《阿凡达》 LOTR 矩阵 海盗》 Matrix Pirates

|        |     |     |
|--------|-----|-----|
| 爱丽丝1   | 0.8 |     |
| 鲍勃     | 0.5 | 0.3 |
| 卡罗尔0.9 | 1   | 0.8 |
| 大卫     | 1   | 0.4 |

在实践中，我们发现物品-物品通常比用户-用户效果  
更好。  
。为什么？课堂问题（发送到：[bigdatacomputing@163.com](mailto:bigdatacomputing@163.com)）

# 协同过滤的优缺点

---

- **+适用于任何类型的物品**
  - 无需特征选择
- **-冷启动：**
  - 系统中用户数量不足，无法找到匹配对象
- **-稀疏性：**
  - 用户/评分矩阵是稀疏的
  - 很难找到对相同商品进行过评价的用户
- **-第一评级人：**
  - 不推荐之前未评分的项目
  - 新物品、深奥物品
- **-流行度偏差：**
  - 无法向有独特品味的人推荐商品
  - 倾向于推荐热门商品

# 混合方法

---

- 实现两个或多个不同的推荐器，并结合预测
  - 可能使用线性模型
- 在协同过滤中加入基于内容的方法
  - 新项目问题的项目概要
  - 应对新用户问题的人口统计信息

# 备注和实用提示

---

- 评价
- 错误度量
- 复杂性/速度



# 评价

---

电影

用户

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 3 | 4 |   |   |   |
|   | 3 | 5 |   |   | 5 |
|   |   | 4 | 5 |   | 5 |
|   |   | 3 |   |   |   |
|   |   | 3 |   |   |   |
| 2 |   |   | 2 |   | 2 |
|   |   |   |   | 5 |   |
|   | 2 | 1 |   |   | 1 |
|   | 3 |   |   | 3 |   |
| 1 |   |   |   |   |   |

# 评价

---

电影

用户

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 3 | 4 |   |   |   |
|   | 3 | 5 |   |   | 5 |
|   |   | 4 | 5 |   | 5 |
|   |   | 3 |   |   |   |
|   |   | 3 |   |   |   |
| 2 |   |   | ? |   | ? |
|   |   |   |   | ? |   |
|   | 2 | 1 |   |   | ? |
|   | 3 |   |   | ? |   |
| 1 |   |   |   |   |   |

测试数据集

# 评估预测

---

## □ 将预测与已知评分进行比较

### ■ 均方根误差 (RMSE)

- $\sqrt{\sum_i (r_{xi} - r^*_{xi})^2 / N}$ , 其中 $r_{xi}$ 是预测的,  $r^*_{xi}$ 是真实的评级  
x on i, N是预测的项目数

### ■ 前10名的精确度:

- 前10名的百分比

### ■ 秩相关:

- 系统和用户完整排名之间的斯皮尔曼相关性

## □ 另一种方法: 0/1模型

### ■ 覆盖

- 系统可以进行预测的项目/用户数量

### ■ 准确率

- 预测的准确性

### ■ 受试者工作特征 (ROC)

- 假阳性率与真阳性率之间的权衡曲线

# 误差度量问题

---

- 过分关注准确性有时会忽略重点
  - 预测多样性
  - 预测背景
  - 预测顺序
- 在实践中，我们只关心预测高评级：
  - RMSE可能会惩罚一个在高评分方面表现良好的方法，而在其他方面表现糟糕的方法

# 协同过滤：复杂性

---

- 昂贵的步骤是找到 $k$ 个最相似的客户： $O(|X|)$
- 运行时成本太高
  - 是否可预计算
- 朴素的预计算需要 $O(k \cdot |X|)$  时间
  - $X$ ..客户群
- 我们已经知道怎么做！
  - 高维近邻搜索 (LSH)
  - 聚类
  - 降维

# 提示：添加数据

---

## □ 充分利用所有数据

- 不要试图减少数据大小，以使复杂的算法工作
- 在大数据上，简单的方法效果最好

## □ 添加更多数据

- 例如，添加关于流派的IMDB数据

## □ 更多的数据胜过更好的算法

<http://anand.typepad.com/datawocky/2008/03/more-data-usual.html>

# 致谢

---

## □ 幻灯片改编自：

- Jeffrey D. Ullman教授
- Anand Rajaraman博士
- Jure Leskovec博士